



SYNARTESIS

Every command,
in the order
you meet them

A walkthrough of every command,
against a bench you can run.
Version **0.6.17** · September 2026

Before you start

A bench is set up at `~/syn-check` with three files worth breaking, a policy already written and pinned, and a stand-in agent so you can drive the proxy without wiring a real AI client.

Work down this document. **Each step leaves the state the next one needs** — approvals only exist once something is held, and undo only means something once something has changed.

THE BENCH

Everything below runs from one directory. Go there once and you never type `--manifest` or `--journal` again: Synartesis walks up from where you are, the way `git` finds its root.

```
$ cd ~/syn-check
$ ls
agent    node_modules  package.json  reset  synartesis.yaml  work/
```

THING	WHAT IT IS
<code>work/</code>	<code>report.txt</code> , <code>ledger.csv</code> , <code>notes.md</code> — three files the agent may touch, and nothing outside this folder.
<code>synartesis.yaml</code>	The policy. Written by <code>init</code> , pinned to this exact server build.
<code>./agent</code>	Stands in for Claude or Cursor. Each argument is one tool call. <code>WORK</code> in a path expands to <code>~/syn-check/work</code> .
<code>./reset</code>	Puts the files back and deletes the journal. Run it any time you want a clean start.
<code>node_modules/</code>	The bench's own copy of the filesystem server, so nothing outside this folder can break it.

CHECK YOU ARE ON THE RIGHT BUILD

```
$ synartesis --version
0.6.17
```

`-v`, `version`, `--help`, `-h` and `help` all answer too.

Write a policy

01 · INIT

Starts a server, asks it what it can do, and writes a policy covering every tool. Where a finished policy ships for that server, it uses it. This is already done for the bench — run it to see what it says, or point it at a different server.

```
$ synartesis init fs -- node /path/to/server ~/syn-check/work

Recognised 14 tools, so the policy that ships for filesystem was used.
Read it before you trust it, then point your MCP client at:
```

Everything after `--` is the command that starts the server, exactly as you would type it yourself. `fs` is just the name its tools get prefixed with.

What a policy says about each tool

CLASS	MEANING
readonly	A lookup. Nothing changes, nothing to undo.
reversible	The state before the write is captured, and put back on undo.
compensable	A second action offsets the first. Not an exact rewind.
irreversible	No way back. Held until a person says yes.

A tool no policy mentions is treated as irreversible and held. Synartesis fails closed: a server that grows a new tool tomorrow cannot quietly use it.

02 · CHECK

Starts every server the policy names and confirms the tools it calls actually exist. Run it after every edit. It writes nothing and touches no journal.

```
$ synartesis check

servers  fs
         fs checked against the real server
policies 10 readonly, 2 reversible, 2 irreversible
guarded  2
pinned   fs (14)
```

"checked against the real server" is the policy's own claim about how far it has been tested. The four policies that ship all say `live` except `github`, which says it has never met a real account — and `check` tells you so before it connects.

What it proves. The policy and the server agree today. A mistyped tool name in a snapshot is otherwise indistinguishable at run time from a file that is not there.

03 · PIN

A policy is a claim about what a tool does, anchored to its name — and a name is a weak anchor. A server upgrade can keep `write_file` and change what it takes, leaving your policy confidently describing something else. Pins make that stop the proxy instead.

```
$ synartesis pin

pins:
  fs:
    write_file: "sha256:ce17c85e8a58835..."
    edit_file:  "sha256:88459ef670b139a..."

Paste this into the manifest.
```

It prints rather than editing your file. Pinning is you vouching for what a tool does today; a command that rewrote the policy for you would let that happen with nobody reading it. **The bench is already pinned** — `check` says `pinned fs (14)`.

See it bite

```
$ sed -i '' 's/write_file: "sha256:ce17/write_file: "sha256:dead/' synartesis.yaml
$ synartesis check
synartesis: a pinned tool no longer matches the policy written for it:
  fs.write_file no longer has the shape it was pinned at...

$ sed -i '' 's/write_file: "sha256:dead/write_file: "sha256:ce17/' synartesis.yaml
```

What it proves. The proxy refuses to start on a tool whose shape has moved, rather than serving a policy that no longer describes it. That failure is the one that produces a confident, wrong undo.

Run it

04 · PROXY

You almost never type this. It speaks MCP on stdin and stdout, so a banner on that stream would corrupt it — your AI client starts it, not you. It is here so you know what `install` is wiring up.

```
$ synartesis proxy --manifest ~/syn-check/synartesis.yaml
```

FLAG	WHY
<code>--server fs</code>	Expose only one server from a policy that describes several.
<code>--http 8787 --token <secret></code>	For a client that cannot start a process. Binds <code>127.0.0.1</code> only and refuses anything without the bearer token.
<code>--journal <path></code>	Where the record goes. Found automatically otherwise.

On the token. It is a shared secret, so treat it like a password: keep it out of a shell history you share, and prefer a tunnel that does its own authentication over exposing the port.

05 · MAKE IT DO THINGS

`./agent` sends tool calls through the proxy exactly as a real client would. Run all three of these — the rest of this document depends on the state they leave.

```
$ ./agent \  
'write_file {"path":"WORK/report.txt","content":"Q3 CLOSE\nnorth  CLOSED\nsouth  CLOSED\n"}' \  
'edit_file {"path":"WORK/notes.md","edits":[{"oldText":"Waiting on the east figures.","newText":"The quarter is closed."}]}\' \  
'write_file {"path":"WORK/summary.txt","content":"all regions closed\n"}'  
  
called  write_file  Successfully wrote to ../report.txt  
called  edit_file   ```diff Index: ../notes.md ...  
blocked write_file  Synartesis is holding this call for approval, because  
                        nothing was captured to restore...
```

Two writes went through. **The third was held.** `summary.txt` does not exist yet, so there is no prior state to put back, and the filesystem server has no delete. Creating a file is genuinely unundoable here, so it asks.

What it proves. The gate is not a list of scary-sounding tool names. The same `write_file` went through twice and was held once, decided by whether a before-image could actually be captured.

Look

06 • LIST

One line per session, newest first.

```
$ synartesis list

session          started    status    actions  agent
14e8eacc-689d-4195-869e-0511b71d3a3c 22:08:26 complete 3 agent (1 awaiting approval)
```

Add `--json` for a machine. The first eight characters of a session id are enough to name it anywhere.

07 • SHOW

The timeline, with the undo recorded for each step. Called with no id it takes the newest.

```
$ synartesis show

1 <- reversible applied fs.write_file
  path ../work/report.txt content Q3 CLOSE north CLOSED...
  undo: path ../work/report.txt content Q3 CLOSE north OPEN...
2 <- reversible applied fs.edit_file
3 <- reversible gated fs.write_file
  note: nothing was captured to restore, so this cannot be undone
```

COMMAND	WHAT CHANGES
---------	--------------

<code>show <id> --full</code>	Every argument, snapshot and inverse in full, nothing elided.
-------------------------------------	---

<code>show <id> --live</code>	Reads each file <i>as it is now</i> and says which still match. Costs real reads, so it is opt-in.
-------------------------------------	--

<code>show <id> --json</code>	The same, for a machine.
-------------------------------------	--------------------------

Now change a file by hand and look again. This is the check worth knowing about:

```
$ echo "east REOPENED" >> work/report.txt
$ synartesis show --live

1 <- reversible applied fs.write_file changed since
```

What it proves. Synartesis records what the *agent* did, not what happened to the file. Your own edit never went through the proxy — which is exactly what lets it notice you.

08 • WATCH, AND THE SCREEN

Two live views. Leave either running in a second terminal while an agent works.

```
$ synartesis watch
```

```
watching 1 run, 0 live - 3 recent actions - 1 waiting  
22:08:26 fs write_file report.txt done, can undo  
22:08:26 fs write_file summary.txt waiting for you
```

```
$ synartesis
```

enter opens, **u** undoes, **p** previews, **l** checks the world now, **f** expands, **g** shows what is held, **c** lists every AI on the machine, **q** quits. Everything in this document can be done from here.

Decide

09 • GATES

Held calls, with the exact arguments — not the agent’s description of them.

```
$ synartesis gates

2a1f9c3d-... 2026-09-14T22:08:26Z
fs.write_file {"path":"../work/summary.txt","content":"all regions closed\n"}
reversible nothing was captured to restore, so this cannot be undone —
            the read said: ENOENT: no such file or directory

synartesis approve 2a1f9c3d --by <name>
```

What it proves. You approve what will actually be sent, shown verbatim, with the server’s own words about why it is held. A model’s summary of its own request is not what you are agreeing to.

10 • APPROVE

Approving records a decision; it does not send the call. The agent tries again and it goes through.

```
$ synartesis approve 2a1f9c3d --by arhaan
approved fs.write_file 2a1f9c3d-...

$ ./agent 'write_file {"path":"WORK/summary.txt","content":"all regions closed\n"}'
called write_file Successfully wrote to ../summary.txt
```

`--by` is who said yes; it defaults to your username. `--all` answers everything waiting at once. An approval covers **that call**, not every call that looks like it — approving once does not authorise it forever.

Look at the first session again

```
$ synartesis show <first id>

3 <- reversible used fs.write_file
approved by arhaan at today at 22:09:01
note: approval was used by action 782b63d3-...
```

What it proves. The row reads **used**, not *denied*, and credits you with the approval you actually gave. Until this build it said “denied by arhaan” beside a call arhaan had just approved and which had gone through.

11 • DENY

The other answer. The call never goes out, and the reason is recorded.

```
$ ./agent 'write_file {"path":"WORK/invoice.txt","content":"send it\n"}'  
blocked write_file ...holding this call for approval  
  
$ synartesis deny --all --by arhaan --reason "not this one"  
denied fs.write_file ...  
  
$ ls work/invoice.txt  
ls: work/invoice.txt: No such file or directory
```

`--reason` is free text and ends up in the timeline. The agent is told it was refused and why.

Undo

12 • UNDO --DRY-RUN

Plans the whole reversal and sends nothing. Do this before every real undo you care about.

```
$ synartesis undo <first id> --dry-run

3 skip    fs.write_file  never applied: its approval moved to the call that ran
2 revert  fs.edit_file   state matches; applying inverse
    would call fs.write_file path ../work/notes.md  content # Quarter notes Waiting...
1 revert  fs.write_file  state matches; applying inverse

R E S U L T  rolled_back
```

It walks backwards — newest first — because putting an older state back before a newer one would leave you with neither.

13 • UNDO

The whole point. With no id it takes the most recent session and says which.

```
$ synartesis undo <first id>

2 revert  fs.edit_file   state matches; applying inverse
1 revert  fs.write_file  state matches; applying inverse

R E S U L T  rolled_back

$ cat work/report.txt
Q3 CLOSE — REGIONAL STATUS
north    OPEN
south    OPEN
```

What it proves. The file is back to the bytes it held before the agent touched it — not a diff applied in reverse, the actual prior contents, read and stored before the write went out.

`summary.txt` stays. It was a creation with nothing to restore and no delete to call, so undo reports it as **permanent**, leaves it, and marks the session **partial** rather than claiming success.

14 • UNDO --TO

Undo everything down to a sequence number and stop. For when the agent's first few steps were right and it went wrong later.

```

$ ./reset
$ ./agent \
'write_file {"path":"WORK/report.txt","content":"one\n"}' \
'write_file {"path":"WORK/ledger.csv","content":"two\n"}' \
'write_file {"path":"WORK/notes.md","content":"three\n"}'

$ synartesis undo --to 3

    3 revert    fs.write_file  state matches; applying inverse
left alone
    2 kept      fs.write_file  below --to 3, so it is left as it is
    1 kept      fs.write_file  below --to 3, so it is left as it is

R E S U L T  partial

```

Steps 1 and 2 stay changed; step 3 is back. The result says **partial**, and the steps it deliberately left are listed rather than silently skipped.

15 · DRIFT, AND --FORCE

This is the part most worth testing. If *you* edited a file after the agent did, putting the old version back would destroy your work. Synartesis checks first.

```

$ ./reset
$ ./agent 'write_file {"path":"WORK/ledger.csv","content":"region,amount\nnorth,0\nsouth,0\n"}'
$ echo "east,999" >> work/ledger.csv
$ synartesis undo

    1 halt      fs.write_file  drift detected

halted at 1  drift detected
nothing was written here

drift at sequence 1: the resource is not in the state this run left it in.
+ east,999

undoing anyway would write:
- north,0   - south,0   - east,999
+ north,412800 + south,288400

```

It shows you **both** diffs: what changed since, and what undoing would write over. Exit code 1, and nothing touched.

COMMAND	WHAT IT MEANS
<i>(nothing)</i>	Keep the change, drop the undo.
<code>undo --replan</code>	You put the file back yourself; rebuild each undo from the current policy and try again.
<code>undo --force</code>	Print exactly what would be written over, and stop.
<code>undo --force --yes</code>	Go ahead and lose the change.

```
$ synartesis undo --force
nothing has been written. To go ahead and lose that:
synartesis undo 14e8eacc --force --yes

$ synartesis undo --force --yes
1 revert fs.write_file the resource had changed since; that change was
overwritten [unverified]
```

What it proves. Refusing your own undo is the hard part of this product, and it is a two-step ask even when you mean it. It also tells you afterwards that the step was `[unverified]` — it did what you said, and will not pretend it was safe.

16 • CLOSE

A proxy that is killed rather than disconnected leaves its session active. Nothing can tell that apart from a session still running, so it is asked for rather than guessed.

```
$ synartesis close
nothing is open; every run has ended cleanly
```

That is the ordinary answer and it exits `0`. Name a session — `synartesis close 14e8eacc` — to close a specific one.

17 • PRUNE

The journal holds a copy of your file contents. That is what makes undo work, and it is why it grows. Pruning throws old sessions away, and with them the ability to undo them.

```
$ synartesis prune --dry-run
The journal is 48 kB. Runs still active, and any holding a call that is
waiting or in flight, are never pruned.

$ synartesis prune --older-than 0 --dry-run
3 runs and 6 actions would go. Nothing was changed.
```

Default is 30 days. Always look with `--dry-run` first: this is the one command here that destroys the thing the product exists to give you.

Wire it up

18 · INSTALL, AND STATUS

Finds what Claude Code, Claude Desktop, Cursor and Codex already list, writes a policy covering all of it, and points each entry at the proxy. **This edits your real config files**, so look first.

```
$ synartesis install --dry-run

Claude Desktop global
  filesystem      the policy that ships for filesystem (14 tools)
Codex global
  node_repl       drafted, every tool held until you say how to undo it

Nothing was written. Run without --dry-run to apply.
```

COMMAND	WHAT IT DOES
<code>install --dry-run</code>	Says what it would do. Writes nothing.
<code>install --print</code>	Prints the config entries so you can paste them yourself.
<code>install --client cursor</code>	Just one client.
<code>install</code>	Does it. Your original config is copied aside first.
<code>status</code>	What is covered, what is not, and how big the journal is.
<code>uninstall</code>	Puts the original configs back.

Restart the client afterwards. Claude Desktop and Cursor read their config at startup. Until you quit and reopen, nothing goes through the proxy and the journal stays empty — which looks exactly like a broken install.

19 · DESKTOP

A separate download. Talk to any model — Claude, Gemini, anything OpenAI-compatible, or a local model with no API key — and watch every tool call be recorded and reversed as it happens. Same journal, same undo engine as everything above.

```
$ synartesis desktop
opening /Applications/Synartesis.app
```

If it is not installed, it tells you where to get it rather than failing. The npm package does not include it: shipping a browser engine inside every install would be a poor trade.

20 · FLAGS AND EXIT CODES

FLAG	MEANING
<code>--manifest <path></code>	The policy. Found by walking up from where you are, then <code>~/.synartesis</code> .
<code>--journal <path></code>	The record. Found the same way.
<code>--json</code>	On <code>list</code> , <code>show</code> and <code>gates</code> .
<code>SYNARTESIS_HOME</code>	Environment variable; moves where both are looked for.
<code>SYNARTESIS_NO_HINTS</code>	Environment variable; turns off the next-step line described below.

The last line of most commands

Nearly every command ends by naming the one thing worth doing next, with the session id already filled in:

```
local-agent-mode-filesystem changed something 6d ago: synartesis show 7f1dc7dd
```

If you gave `--journal` or `--manifest`, the line carries them too, so it can be pasted from any directory and still mean the session you were looking at.

It is worked out from the journal, not from what you typed, so it changes as the state does — a call held for approval outranks everything else, and a session in which nothing was written is never offered for undo. There is at most one, it is absent from `--json`, and `SYNARTESIS_NO_HINTS` turns it off.

A mistyped command gets the same treatment: `synartesis lst` answers `did you mean list?` and five commands, rather than the whole help page.

CODE	MEANING
0	Did what you asked.
1	Halted or refused — drift, an unknown outcome, a failed reversing call.
2	Bad usage, or a policy or journal it could not use.

Start over at any point

```
$ ./reset
bench reset. no journal, files at baseline.
```

Every command in this document was run against the real filesystem MCP server before it was written down.